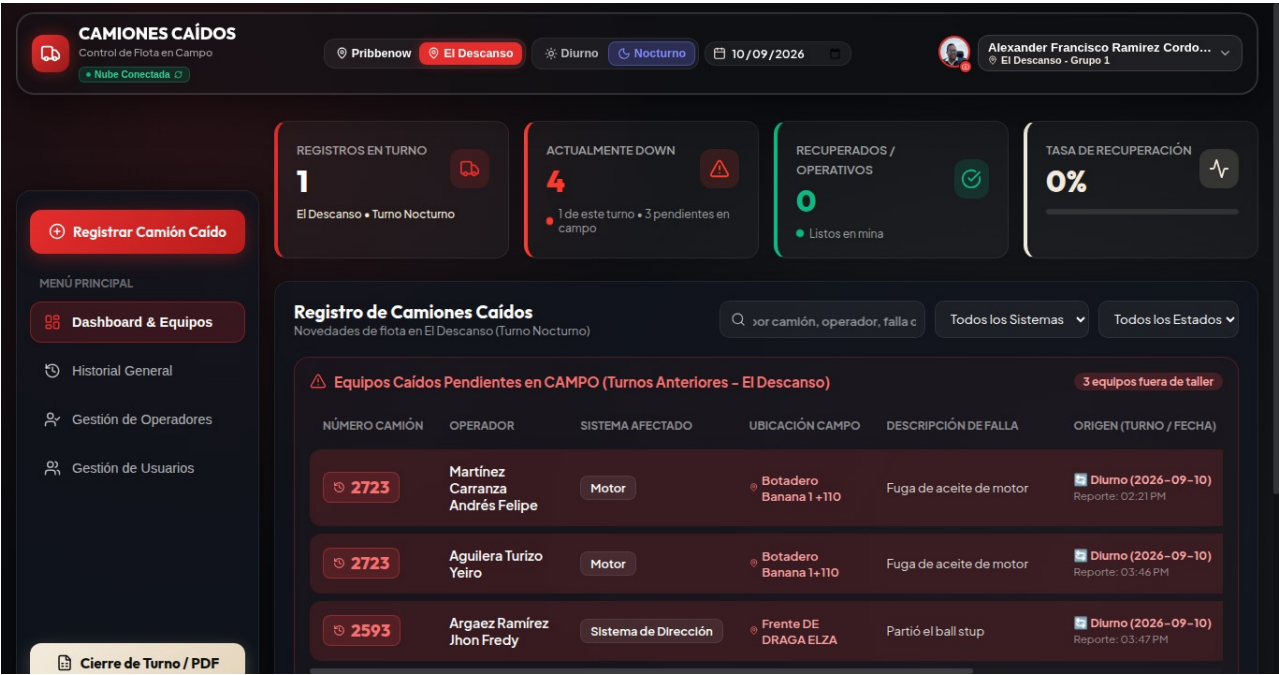


Camiones Caídos

Manual técnico

Arquitectura notificaciones y recuperación

Mina Pribbenow y Mina El Descanso
Versión documental 1.1 | 27 de septiembre de 2026



Índice de Secciones Técnicas

1 Arquitectura y alcance.....	3
2 Dependencias y herramientas.....	3
3 Mapa del repositorio.....	3
4 Autenticación y sesiones.....	4
5 Modelo de datos y relaciones.....	4
6 Relación del operador y copia histórica.....	4
7 Políticas y protección de columnas.....	5
8 Administración mediante funciones Edge.....	5
9 Guardado y eventos operacionales.....	6
10 Evaluación de cambio de turno.....	7
11 Scheduler y secretos de ejecución.....	7
12 Emisor Web Push y entrega.....	8
13 Campana y lectura en tiempo real.....	8
14 Suscripciones y aislamiento por cuenta.....	9
15 PWA y Service Worker personalizado.....	10
16 Caché local y trabajo sin conexión.....	10
17 Rendimiento y exportación.....	11
18 Variables y configuración.....	11
19 Desarrollo y verificación local.....	12
20 Publicación y distribución de manuales.....	12
21 Respaldos y recuperación.....	13
22 Diagnóstico de incidentes.....	14
23 Seguridad y mantenimiento.....	14
24 Lista de verificación previa al release.....	15

1 Arquitectura y alcance

Camiones Caídos combina una SPA React distribuida como PWA, Supabase Auth y PostgreSQL con RLS, Storage privado para manuales y funciones Edge en Deno. Las notificaciones tienen dos canales: una fila personal consultada por la campana y un envío Web Push a suscripciones activas.

Este manual describe los componentes del repositorio y el procedimiento para operarlos. Antes de intervenir un entorno, contraste sus funciones, migraciones, políticas, configuración y versión del frontend. Una compilación local no acredita el estado de la infraestructura remota.

2 Dependencias y herramientas

package.json declara React y React DOM ^19.2.8, Vite ^8.2.0, @supabase/supabase-js ^2.109.0, vite-plugin-pwa ^1.3.0, jspdf ^4.2.1, jspdf-autotable ^5.0.8, xlsx ^0.18.5 y oxlint ^1.75.0. Son rangos declarados; package-lock.json determina la resolución reproducible.

El frontend usa JavaScript y JSX; las funciones Edge usan TypeScript. Workbox gestiona precache y navegación del Service Worker. No deduzca la versión de PostgreSQL ni las prestaciones del plan de Supabase a partir de versiones de documentos anteriores.

3 Mapa del repositorio

src/context/AuthContext.jsx gestiona sesión y perfil; ReportContext.jsx gestiona reportes y operadores; NotificationContext.jsx gestiona la campana y Realtime.

src/components/Notifications/ contiene NotificationBell, NotificationPanel, NotificationItem y NotificationDetail. Layout/NavUserProfile.jsx maneja suscripciones, manuales y cierre de sesión. Forms/TruckReportModal.jsx y Dashboard/TruckTable.jsx confirman guardados antes de solicitar avisos.

src/utls/pushUtils.js maneja el ciclo de suscripción; src/sw.js recibe y abre push. Los utilitarios pdfUtils.js y exportUtils.js generan reportes operativos.

supabase/functions/ contiene siete funciones: admin-create-user, admin-reset-password, admin-delete-user, get-doc-url, notify-operational-event, evaluate-shift-carryover y send-web-push. El archivo supabase_schema.sql contiene el esquema base; supabase/migrations/ contiene las ampliaciones. Ninguno sustituye una comprobación del esquema remoto.

docs/user/, docs/admin/ y docs/technical/ contienen los manuales. tests/ contiene pruebas locales con dependencias simuladas.

4 Autenticación y sesiones

El usuario ingresa su cédula y contraseña. AuthContext deriva el correo técnico con el identificador limpio y el dominio camionescaidos.internal; Supabase Auth valida la contraseña y emite la sesión. app_users enlaza el perfil mediante auth_user_id.

El perfil incluye rol, mina, group_name, avatar, must_change_password e is_active. El cliente conserva un perfil validado para tolerar fallos de transporte; una caché no otorga permisos adicionales al servidor.

El cierre desde el menú intenta retirar la suscripción push antes de ejecutar logout. Las esperas push están acotadas y su fallo no impide iniciar el cierre de sesión; una advertencia indica que la limpieza no fue confirmada. No interprete cerrar sesión como evidencia suficiente de eliminación remota de la suscripción.

5 Modelo de datos y relaciones

app_users guarda identidad de aplicación, national_id, name, role, mine, group_name, must_change_password, avatar, created_at, auth_user_id e is_active. operators guarda el censo con id, name, mine, group_name, status y created_at.

truck_reports guarda id, truck_id, mine, shift, date, operator, operator_id, system, detail, location, status, down_time, estimated_return_time, actual_return_time y marcas created_at y updated_at. Los estados operativos manejados por las notificaciones son DOWN y OPERATIVO.

notifications contiene destinatarios user_id y auth_user_id, mina, turno, grupo, operational_date, evaluation_moment, title, message, truck_count, truck_ids, is_read, read_at y created_at. La ampliación agrega event_type, report_id, event_key y metadata.

push_subscriptions vincula user_id y auth_user_id con endpoint único, claves p256dh y auth, user_agent, platform, is_active, last_used_at, created_at y updated_at. platform admite android, ios, desktop y unknown.

Las tablas de notificaciones y suscripciones referencian app_users y auth.users con eliminación en cascada. report_id en notifications es un campo de texto; no suponga una clave foránea donde la migración no la define.

6 Relación del operador y copia histórica

truck_reports.operator_id identifica al operador para cruces analíticos. truck_reports.operator conserva el texto del nombre asociado al registro. Así, el historial y las exportaciones no dependen de resolver el nombre actual mediante un join.

La copia del nombre protege el contexto histórico ante cambios en el censo; no significa que el reporte completo sea inmutable. Antes de modificar operadores o migrar identificadores, concilie referencias y casos históricos sin operator_id.

7 Políticas y protección de columnas

Las funciones `get_auth_user_role()`, `get_auth_user_mine()` e `is_admin()` apoyan las políticas del esquema base. `truck_reports` limita la operación a administradores o usuarios de la mina; la eliminación se restringe a Administrador o Encargado de su mina. `operators` reserva escrituras a administradores.

`notifications_select_own` y `notifications_update_own` limitan lectura y actualización a `auth_user_id = auth.uid()`. No hay políticas de inserción ni eliminación para clientes normales; las funciones backend crean las filas. El trigger protege contenido, destinatarios y metadatos, dejando al cliente el estado de lectura. Al leer se completa `read_at`; al desmarcar se limpia.

`push_subscriptions` permite consultar y gestionar las suscripciones propias. El trigger comprueba en inserción la correspondencia entre `auth_user_id` y `user_id` y protege identidad y fecha de creación en actualizaciones normales.

La migración de `is_active` redefine `trg_protect_app_users_columns`: `id`, `national_id`, `auth_user_id` y `created_at` se comprueban antes del bypass de servicio. Para otros campos restringe cambios de usuarios no administradores, incluido `is_active`. No use la descripción antigua que suponía bypass de identidad para `service_role`.

Revise las cinco migraciones locales en su orden temporal. La primera también actualiza perfiles existentes a `is_active = true`: no la vuelva a ejecutar indiscriminadamente sobre datos operativos. Validar o aplicar migraciones es una intervención separada del mantenimiento de manuales.

8 Administración mediante funciones Edge

`admin-create-user` valida al administrador, rechaza duplicados de cédula o correo técnico, crea Auth y luego el perfil con clave temporal `caidos1234` y `must_change_password`. Si falla el perfil intenta una eliminación compensatoria de Auth; son operaciones separadas, no una transacción atómica entre servicios.

`admin-reset-password` asigna la clave temporal y actualiza `must_change_password` con reintentos. `admin-delete-user` bloquea autoeliminación y eliminación del último administrador, y coordina eliminación de Auth y perfil. Consulte la respuesta de cada operación antes de confirmar éxito.

`get-doc-url` autoriza por rol y firma enlaces de manuales. Las funciones de notificaciones se explican a continuación. La autorización vive en cada handler; no suponga que todas aceptan los mismos roles o cabeceras.

9 Guardado y eventos operacionales

ReportContext confirma inserciones y ediciones mediante la fila devuelta por el servidor. El cambio de estado actualiza estado y horas, filtra por el estado anterior y exige una fila confirmada. Formularios y tabla esperan ese resultado; ante error conservan el formulario o confirmación y no solicitan el aviso de éxito.

notify-operational-event recibe event_type new_report o status_change, report_id y truck_id; para cambios recibe previous_status y new_status. Exige sesión válida, perfil activo y rol Administrador, Encargado o Digitador. Lee el reporte con el cliente del solicitante y verifica además la mina para no administradores.

El camión solicitado debe coincidir con el reporte. Sistema afectado, mina, fecha y turno se toman del registro. En status_change el estado anterior debe ser válido y el nuevo debe coincidir con el persistido; si no cambió, responde sin notificar.

Los destinatarios son perfiles activos con Auth de la mina del reporte y grupo del actor, más administradores activos globales, excluyendo al actor. Se deduplican por auth_user_id. El grupo no se deriva del calendario del turno.

La clave del nuevo reporte usa su identificador; la del cambio usa datos del estado y la marca persistida updated_at, con fallback a created_at. La restricción UNIQUE (user_id, event_key) evita duplicar la misma clave por destinatario. Los datos de una solicitud no constituyen por sí solos evidencia histórica de autoría o de la transición anterior: el flujo no es una transacción de auditoría entre guardado y aviso.

10 Evaluación de cambio de turno

evaluate-shift-carryover admite **POST** con **evaluationMoment** 06:30 o 18:30 y fecha opcional YYYY-MM-DD; sin fecha usa America/Bogota. Autoriza un x-scheduler-secret válido o un JWT de Administrador verificado por el handler.

Calcula el grupo entrante con ciclo de 21 días: 7 diurnos, 3 de descanso, 7 nocturnos y 4 de descanso. El ancla local es 2026-09-17, índices Grupo 1 = 2, Grupo 2 = 16 y Grupo 3 = 9. Confirme con operación esa correspondencia antes de cambiar el calendario.

Evalúa Pribbenow y El Descanso. A las 06:30 selecciona fechas anteriores; a las 18:30, fechas anteriores o turno Diurno de la fecha evaluada. Ordena por created_at descendente y toma el primer reporte por truck_id dentro de ese conjunto; después exige status DOWN y ubicación en campo.

isEquipmentInField excluye taller, abreviaturas tlr o tllr, bahia, bahía y palabras bh o bay. Una ubicación vacía se considera campo. Esta clasificación textual depende de la calidad de la captura.

Si no hay pendientes, responde NO_CARRYOVER para la mina; si no hay destinatarios, NO_ACTIVE_RECIPIENTS. Si hay ambos, crea avisos para usuarios activos de la mina y grupo entrante más administradores activos globales. Usa UNIQUE (user_id, mine, shift, operational_date, evaluation_moment).

Un nuevo reporte del turno entrante queda fuera del conjunto consultado y no sustituye al de turnos anteriores en esta evaluación. Ante discrepancias con el Dashboard, contraste ambos reportes y la fecha operativa antes de proponer un cambio de regla.

11 Scheduler y secretos de ejecución

Las migraciones habilitan pg_cron y pg_net y definen dispatch_shift_carryover_evaluation(p_moment). La función lee scheduler_secret de Vault y envía la cabecera x-scheduler-secret a evaluate-shift-carryover. El valor debe corresponder a SCHEDULER_SECRET de las funciones Edge; nunca se documenta ni imprime su contenido.

La ejecución del dispatcher se revoca a PUBLIC, anon y authenticated y se concede a postgres y service_role. Su SQL contiene una URL fija de proyecto; revísela antes de preparar un entorno de pruebas aislado.

Los trabajos definidos son evaluate-shift-carryover-morning-0630 con expresión 30 11 * * * y evaluate-shift-carryover-evening-1830 con 30 23 * * *. Corresponden a 06:30 y 18:30 Colombia. La migración crea cada trabajo si su nombre no existe; no corrige automáticamente una definición antigua con el mismo nombre.

Un identificador devuelto por pg_net acredita la solicitud encolada, no la creación de notificaciones ni la entrega push. Revise cron.job, cron.job_run_details, respuesta HTTP y logs del handler según el entorno. La configuración de verificación JWT del gateway debe permitir llegar al handler a las solicitudes del scheduler, conservando su validación de secreto.

12 Emisor Web Push y entrega

send-web-push autoriza al scheduler o al Administrador y valida el mensaje. Acepta target_user_ids, target_auth_user_ids, target_subscription_ids o una target_subscription directa con endpoint HTTPS y claves p256dh y auth.

Sin destinatarios explícitos válidos responde HTTP 400 antes de consultar suscripciones o enviar. Las listas provistas deben ser arrays de textos no vacíos; se recortan y deduplican. Varios filtros no vacíos se combinan por intersección. Un destino válido sin coincidencias produce cero envíos, nunca un envío global. Una suscripción directa válida conserva prioridad sobre las listas.

Para listas consulta suscripciones activas. Usa VAPID y registra cantidades de objetivos, éxitos, expirados y fallos. Las respuestas 404 o 410 del servicio push permiten retirar suscripciones expiradas; otros fallos requieren diagnóstico. Un éxito de transporte no certifica lectura ni atención por el usuario.

Los productores solicitan push únicamente para filas de notificación recién insertadas. Si falla el envío después de insertar, repetir la misma clave no reenvía ese push. No existe una cola persistente que garantice reintentos de todos los fallos. La campana puede conservar el aviso aunque el dispositivo nunca muestre el push.

13 Campana y lectura en tiempo real

NotificationContext consulta los 30 avisos más recientes visibles por RLS y se suscribe a INSERT y UPDATE de notifications por Realtime. La migración agrega la tabla a supabase_realtime; la publicación desplegada debe comprobarse.

El contador se deriva de notifications.filter(n => !n.is_read). Repetir un INSERT no debe agregar una segunda tarjeta ni inflar el contador. La interfaz muestra 9+ por encima de nueve. No hay paginación visible del historial completo.

Abrir el detalle solicita marcar ese aviso como leído. Marcar leídas actualiza todos los pendientes visibles por RLS en el servidor, aunque algunos no estén cargados. La UI hace actualización optimista y restaura el estado ante fallo. La lectura no cambia un reporte ni certifica una intervención operativa.

El detalle navega al Dashboard con el contexto de la notificación. La tarjeta representa el momento del evento y puede quedar desactualizada respecto del reporte. La desconexión puede retrasar la actualización; no afirme entrega inmediata ni sincronía garantizada entre todos los dispositivos.

14 Suscripciones y aislamiento por cuenta

pushUtils.js comprueba soporte, permiso, sesión, suscripción del navegador y fila activa asociada a los dos identificadores del usuario. getPushSubscriptionState consulta sin solicitar permisos ni reactivar alertas.

La activación es explícita desde NavUserProfile. Si una suscripción no puede verificarse como propia, el flujo retira la suscripción local y crea una nueva; no transfiere una fila de otra cuenta. Tras upsert verifica el estado antes de indicar Activas.

La desactivación comprueba el resultado del navegador y de la eliminación remota de la cuenta y endpoint. Si solo se completa una parte, conserva contexto para Reintentar desactivación. Ese contexto está en memoria y puede perderse al recargar; la limpieza no es una tarea persistente en segundo plano.

El perfil reevalúa al abrir el menú, recuperar conexión, obtener foco o recibir PUSH_SUBSCRIPTION_CHANGED. Ignora resultados tardíos de otra cuenta. Usa esperas de cinco segundos por operación de consulta o limpieza; el diálogo de permiso del usuario no se fuerza a vencer. Un timeout no demuestra que una transacción remota no se haya completado.

Estados visibles: Activas y Desactivar Alertas; Activar Alertas para configuración pendiente; Reintentar desactivación para limpieza parcial; Comprobar de nuevo ante estado desconocido; mensajes específicos por permiso bloqueado, falta de soporte o instalación necesaria en iPhone.

15 PWA y Service Worker personalizado

vite.config.js usa strategies injectManifest, srcDir src, filename sw.js y registerType autoUpdate. src/sw.js aplica skipWaiting y clientsClaim, precache del manifiesto compilado, limpieza de cachés antiguas y NavigationRoute hacia index.html. El número de activos depende de cada build.

El evento push interpreta el payload y muestra la notificación. Si detecta una ventana visible o enfocada solicita presentación silenciosa y sin vibración. No elimina la notificación nativa: su presentación final depende del navegador y sistema operativo.

notificationclick limita el destino a una ruta del mismo origen, agrega truck_id, mine, shift y date cuando hay metadatos operacionales, y enfoca o abre la app. El push de cambio de turno solo aporta la raíz; su detalle se consulta desde la campana. Pulsar el push no ejecuta markAsRead por sí mismo.

pushsubscriptionchange avisa a ventanas mediante PUSH_SUBSCRIPTION_CHANGED. El perfil comprueba estado y puede requerir activación explícita; no hay autorreparación garantizada si no hay una ventana disponible.

El manifiesto usa nombre Camiones Caídos - Control de Flotas, modo standalone, iconos de 192 y 512 píxeles y tema #0F1115. La activación push requiere soporte del navegador y permiso. El cliente indica agregar a Pantalla de Inicio en iOS cuando corresponda.

16 Caché local y trabajo sin conexión

AuthContext puede conservar el perfil validado ante fallos de transporte y revalidar al recuperar conexión. localStorage usa camiones_user_profile, camiones_reports, camiones_operators y camiones_mine; Supabase administra su clave de sesión sb-<ref>-auth-token.

Los reportes y operadores cargados permiten consulta limitada sin red; no equivalen a una copia completa ni necesariamente actual. No existe una cola de escrituras offline para asegurar altas, ediciones o cambios de estado posteriores. La confirmación requiere respuesta del servidor.

No publique tokens ni contenidos completos de almacenamiento en diagnósticos. Borrar datos del sitio afecta caché, sesión y configuración local: registre primero el incidente y coordine la recuperación del acceso y de las alertas.

17 Rendimiento y exportación

React.lazy carga módulos administrativos e historiales bajo demanda. La configuración manualChunks separa vendor-export-pdf, vendor-export-xlsx, vendor-supabase y vendor-react. Las bibliotecas de exportación se importan dinámicamente donde corresponde.

pdfUtils.js genera reportes con membretes y contenido operativo en el cliente. exportUtils.js produce las hojas Novedades Turno y Pendientes Campo. El nombre del operador se conserva desde el reporte. Compartir archivos depende de la capacidad del dispositivo; descargar permite la alternativa manual.

Los tamaños de bundles, cantidad de entradas precache y tiempos deben medirse en cada versión. Registre dispositivo, red, caché y método si mide rendimiento. Las cifras de la documentación 1.0 son antecedentes, no criterios actuales certificados.

18 Variables y configuración

Frontend: VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY y VITE_VAPID_PUBLIC_KEY. La clave pública VAPID del cliente debe corresponder a VAPID_PUBLIC_KEY del emisor.

Backend: SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY, SCHEDULER_SECRET, VAPID_PUBLIC_KEY, VAPID_PRIVATE_KEY y VAPID_SUBJECT. Vault: scheduler_secret para el dispatcher.

Mantenga valores secretos fuera de documentos, frontend y registros de diagnóstico. SUPABASE_SERVICE_ROLE_KEY y VAPID_PRIVATE_KEY son exclusivamente de servidor. Confirme origen, gateway, CORS y correspondencia de variables sin imprimir secretos.

La existencia de código o nombres de variables en el repositorio no prueba que estén configurados o desplegados. Compruebe el entorno seleccionado antes de ejecutar una app local: puede usar el mismo backend de la operación.

19 Desarrollo y verificación local

npm run dev inicia Vite en el puerto 5173 y escucha en todas las interfaces según package.json. npm run preview sirve el build local. Abrir cualquiera puede conectar con Supabase configurado; acuerde el entorno antes de hacer pruebas funcionales.

npm run build compila dist/ y el Service Worker. npm run lint ejecuta oxlint. node --test tests/*.test.cjs ejecuta las pruebas locales, con Node.js 24 o posterior para stripTypeScriptTypes.

Las pruebas usan memoria para Auth, base de datos, navegador, hooks y transporte push. Cubren permisos, consistencia, persistencia, duplicados, suscripciones y destinatarios. No montan React real ni ejecutan PostgreSQL/RLS, Deno, cron o entrega real. El build del frontend tampoco valida por sí solo las funciones Edge.

Registre el resultado de cada versión y las advertencias aceptadas. No trate cifras antiguas de pruebas, lint o build como una certificación permanente.

20 Publicación y distribución de manuales

El frontend usa los rewrites SPA de vercel.json para dirigir rutas a index.html. Funciones, migraciones, secretos y jobs de Supabase tienen un ciclo de publicación separado. Cualquier despliegue requiere una versión revisada, pruebas y autorización.

get-doc-url sirve documentos del bucket privado docs-private:

user/MANUAL_USUARIO_CAMIONES_CAIDOS.pdf y

admin/MANUAL_ADMINISTRATIVO_CAMIONES_CAIDOS.pdf. Digitador solicita el de usuario;

Administrador y Encargado pueden solicitar ambos. El técnico no está expuesto por ese menú.

Actualizar docs/ no reemplaza los objetos de Storage. Tras aprobación de publicación, respalde los objetos anteriores, cargue solo los PDF autorizados en sus rutas y compruebe descarga, contenido y permisos con cuentas acordadas. El enlace firmado es temporal; solicite uno nuevo desde el menú cuando sea necesario.

Mantenga texto, orden y capturas coherentes entre Word, Markdown y PDF. Exporte PDF desde el Word revisado y valide que no desaparezcan párrafos o tablas. No use una conversión antigua como fuente de la próxima versión sin conciliar diferencias.

21 Respalos y recuperación

Separe respaldo de código, esquema, configuración y datos. Git guarda archivos versionados; no respalda por sí solo datos de Supabase ni secretos. Conserve supabase_schema.sql y migraciones, y documente configuración sin exponer valores.

Los documentos previos citan snapshots del 10 de septiembre de 2026 y copias automáticas. Antes de depender de ellos, verifique su existencia, integridad, fecha, cobertura, retención y capacidad de restauración en el plan real. No dé por garantizadas copias diarias por aparecer en un manual.

No hay cron de respaldo de datos en el frontend. Antes de una restauración, preserve evidencia, acuerde alcance y valide un respaldo en un entorno aislado. Restaurar datos o revertir infraestructura requiere autorización; no sustituya operación con un snapshot antiguo sin conciliación.

22 Diagnóstico de incidentes

Pantalla blanca o ChunkLoadError: registre consola y versión, contraste el Service Worker y los assets del despliegue. Pruebe recarga y un perfil limpio acordado antes de borrar almacenamiento; verifique carga y acceso después.

Build o despliegue fallido: reproduzca `npm run build` y revise las variables del entorno. Si se necesita revertir, seleccione una versión conocida y coordine la autorización; confirme frontend y backend compatibles.

Error de red de Supabase: distinga cobertura local de incidente del servicio. Conserve consulta cacheada cuando sea posible y verifique recuperación con lectura. No reinicie ni restaure el proyecto como primer paso sin diagnóstico.

HTTP 401 o 403: verifique sesión, identidad, perfil y autorización específica del handler. Un 403 no implica siempre falta de rol Administrador. Para problemas RLS, contraste `auth_user_id`, mina y política aplicable; no quite políticas para probar.

Error 42501: revise qué columna o política rechazó la operación. Ante fallos CORS, examine preflight y `_shared/cors.ts`. No exponga credenciales para resolver el diagnóstico.

Aviso ausente de campana: compruebe persistencia del reporte, respuesta del productor, elegibilidad, filas creadas y Realtime. Si solo falta push, revise suscripción activa, VAPID y resultado del emisor. HTTP 400 por destinos vacíos es un rechazo esperado, no motivo para eliminar el filtro.

Alerta de turno ausente: revise jobs, dispatcher, Vault, secreto del handler, respuesta `pg_net`, grupo calculado y pendientes elegibles. Un job ejecutado o `NO_CARRYOVER` no es un comprobante de entrega.

Caché antigua o pérdida de datos: preserve evidencia antes de limpiar o restaurar. Diferencie datos no visibles por filtros de borrado real; acuerde un procedimiento de recuperación antes de escribir en la base de operación.

23 Seguridad y mantenimiento

Revise con especial cuidado `AuthContext`, `ReportContext`, `NotificationContext`, `pushUtils`, `NavUserProfile`, `src/sw.js`, las funciones `Edge` y las migraciones. Cambios de sesiones, permisos o notificaciones pueden afectar datos y destinatarios reales.

Mantenga `service_role` fuera del frontend, verifique aislamiento por usuario y mina, y preserve las restricciones de identidad. La recepción global de avisos para administradores no concede lectura de campanas ajenas mediante las políticas personales.

Revise periódicamente errores de funciones, ejecución de jobs, rechazos push y suscripciones expiradas. Actualice dependencias con evaluación y pruebas; `npm audit` necesita red y no modifica por sí mismo el código salvo comandos adicionales.

Los conteos históricos de 22 usuarios, 814 operadores y 347 reportes corresponden a documentación de septiembre de 2026; no representan inventario actual. Un release requiere evidencia fechada del entorno que se autorice a verificar.

24 Lista de verificación previa al release

1. Auditar el estado local y remoto autorizado, identificar la versión candidata y presentar riesgos pendientes. Separar cambios aprobados de nuevos hallazgos.
2. Ejecutar pruebas locales, lint y build; revisar diffs, manuales y estrategia de reversión. Documentar límites de las simulaciones.
3. Acordar entorno aislado, usuarios y destinatarios antes de pruebas que escriban datos o envíen avisos. Verificar permisos por mina, guardado fallido, contador, lectura y cambio de cuenta.
4. **Probar push en dispositivos acordados:** permiso, instalación, primer y segundo plano, apertura, desactivación parcial, reconexión y logout. Evaluar los cortes del turno sin provocar alertas a la operación.
5. Verificar migraciones, RLS, Realtime, funciones, gateway, VAPID, Vault y cron del destino. Confirmar respaldos y recuperación antes de intervenirlo.
6. Presentar evidencia y obtener autorización separada para commit, push, publicación de manuales y despliegues. No deducir esas autorizaciones de la aprobación de cambios locales.
7. Tras un despliegue autorizado, comprobar los flujos y descargas con el alcance acordado; conciliar resultados y resolver bloqueos antes de autorizar el release final.